

50+ INFOSYS CODING QUESTIONS WITH C++ SOLUTIONS

Created by SYNTAX ERROR | Abhishek Rathor

Each solution includes:

- Problem Statement
- Approach/Algorithm
- Code Implementation
- Time & Space Complexity
- Test Cases with Output

// ===== SECTION 1: ARRAYS & STRINGS =====

/*

Question 1: Find the Largest Element in an Array

Problem: Given an array of integers, find and return the largest element.

Approach: Iterate through array and track maximum element

Time Complexity: $O(n)$

Space Complexity: $O(1)$

*/

```
int findLargest(int arr[], int n) {
```

```
    int maxVal = INT_MIN;
```

```
    for(int i = 0; i < n; i++) {
```

```
        if(arr[i] > maxVal) {
```

```
            maxVal = arr[i];
```

```
        }
```

```

    }

    return maxVal;
}

void testQ1() {

    cout << "\n===== Question 1: Find Largest Element =====\n";

    int arr[] = {3, 7, 2, 9, 1, 5};

    int n = 6;

    cout << "Input: [3, 7, 2, 9, 1, 5]\n";

    cout << "Output: " << findLargest(arr, n) << "\n";

    cout << "Expected: 9\n";

}

/*

```

Question 2: Reverse an Array

Problem: Reverse the given array in-place.

Approach: Use two-pointer technique - swap elements from start and end

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```

*/

void reverseArray(int arr[], int n) {

    int start = 0, end = n - 1;

    while(start < end) {

        swap(arr[start], arr[end]);
    }
}

```

```

        start++;

        end--;

    }

}

void testQ2() {

    cout << "\n===== Question 2: Reverse Array =====\n";

    int arr[] = {1, 2, 3, 4, 5};

    int n = 5;

    cout << "Input: [1, 2, 3, 4, 5]\n";

    reverseArray(arr, n);

    cout << "Output: [";

    for(int i = 0; i < n; i++) {

        cout << arr[i];

        if(i < n-1) cout << ", ";

    }

    cout << "]\n";

    cout << "Expected: [5, 4, 3, 2, 1]\n";

}

```

Question 3: Check if String is Palindrome

Problem: Check if a given string reads the same forwards and backwards

Approach: Compare characters from both ends moving towards center

Time Complexity: $O(n)$

Space Complexity: $O(1)$

*/

```
bool isPalindrome(string s) {  
  
    int left = 0, right = s.length() - 1;  
  
    while(left < right) {  
  
        // Skip non-alphanumeric characters  
  
        while(left < right && !isalnum(s[left])) left++;  
        while(left < right && !isalnum(s[right])) right--;  
  
        if(tolower(s[left]) != tolower(s[right])) {  
            return false;  
        }  
  
        left++;  
        right--;  
    }  
  
    return true;  
}
```

```
void testQ3() {  
  
    cout << "\n===== Question 3: Check Palindrome =====\n";  
  
    string s = "A man, a plan, a canal: Panama";  
  
    cout << "Input: " << s << "\n";  
  
    cout << "Output: " << (isPalindrome(s) ? "true" : "false") << "\n";  
}
```

```

        cout << "Expected: true\n";
    }

/*

```

Question 4: Find Second Largest Element

Problem: Find the second largest element in the array

Approach: Track first and second maximum in single pass

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```

*/

int secondLargest(int arr[], int n) {
    if(n < 2) return -1;

    int first = INT_MIN, second = INT_MIN;

    for(int i = 0; i < n; i++) {
        if(arr[i] > first) {
            second = first;
            first = arr[i];
        }
        else if(arr[i] > second && arr[i] != first) {
            second = arr[i];
        }
    }
}

```

```

        return (second == INT_MIN) ? -1 : second;
    }

void testQ4() {
    cout << "\n===== Question 4: Second Largest Element =====\n";

    int arr[] = {12, 35, 1, 10, 34, 1};

    int n = 6;

    cout << "Input: [12, 35, 1, 10, 34, 1]\n";

    cout << "Output: " << secondLargest(arr, n) << "\n";

    cout << "Expected: 34\n";

}

/*

```

Question 5: Remove Duplicates from Sorted Array

Problem: Remove duplicates in-place from sorted array, return new length

Approach: Two pointer technique - one for reading, one for writing

Time Complexity: $O(n)$

Space Complexity: $O(1)$

*/

```

int removeDuplicates(int arr[], int n) {

    if(n == 0) return 0;

    int j = 0;

    for(int i = 1; i < n; i++) {

```

```

        if(arr[i] != arr[j]) {

            j++;

            arr[j] = arr[i];

        }

    }

    return j + 1;

}

```

```

void testQ5() {

    cout << "\n===== Question 5: Remove Duplicates =====\n";

    int arr[] = {1, 1, 2, 2, 3, 4, 4, 5};

    int n = 8;

    cout << "Input: [1, 1, 2, 2, 3, 4, 4, 5]\n";

    int newLen = removeDuplicates(arr, n);

    cout << "Output: [";

    for(int i = 0; i < newLen; i++) {

        cout << arr[i];

        if(i < newLen-1) cout << ", ";

    }

    cout << "], Length: " << newLen << "\n";

    cout << "Expected: [1, 2, 3, 4, 5], Length: 5\n";

}

```

/*

Question 6: Find Missing Number (1 to n)

Problem: Given array of n-1 integers in range [1,n], find missing number

Approach: Use sum formula: $\text{Sum}(1 \text{ to } n) - \text{Sum}(\text{array elements})$

Time Complexity: $O(n)$

Space Complexity: $O(1)$

*/

```
int findMissing(int arr[], int n) {  
    int totalSum = (n + 1) * (n + 2) / 2;  
    int arraySum = 0;  
  
    for(int i = 0; i < n; i++) {  
        arraySum += arr[i];  
    }  
  
    return totalSum - arraySum;  
}  
  
void testQ6() {  
    cout << "\n===== Question 6: Find Missing Number =====\n";  
    int arr[] = {1, 2, 4, 5, 6};  
    int n = 5;  
    cout << "Input: [1, 2, 4, 5, 6], n=6\n";  
    cout << "Output: " << findMissing(arr, n) << "\n";  
}
```



```
    cout << "Expected: 3\n";  
}
```

```
/*
```

Question 7: Rotate Array by K Positions

Problem: Rotate array to the right by k positions

Approach: Reverse entire array, then reverse first k and last n-k elements

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```
*/
```

```
void rotateArray(int arr[], int n, int k) {
```

```
    k = k % n;
```

```
    // Reverse entire array
```

```
    reverse(arr, arr + n);
```

```
    // Reverse first k elements
```

```
    reverse(arr, arr + k);
```

```
    // Reverse remaining elements
```

```
    reverse(arr + k, arr + n);
```

```
}
```

```
void testQ7() {
```

```
    cout << "\n===== Question 7: Rotate Array =====\n";
```

```
    int arr[] = {1, 2, 3, 4, 5, 6, 7};
```

```

int n = 7, k = 3;

cout << "Input: [1, 2, 3, 4, 5, 6, 7], k=3\n";

rotateArray(arr, n, k);

cout << "Output: [";

for(int i = 0; i < n; i++) {

    cout << arr[i];

    if(i < n-1) cout << ", ";

}

cout << "]\n";

cout << "Expected: [5, 6, 7, 1, 2, 3, 4]\n";

}

/*

```

Question 8: Count Vowels in String

Problem: Count number of vowels (a,e,i,o,u) in a string

Approach: Iterate through string and check each character

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```

*/

int countVowels(string s) {

    int count = 0;

    for(char c : s) {

        c = tolower(c);

        if(c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {

```

```

        count++;
    }

}

return count;
}

void testQ8() {

    cout << "\n===== Question 8: Count Vowels =====\n";

    string s = "Hello World";

    cout << "Input: " << s << "\n";

    cout << "Output: " << countVowels(s) << "\n";

    cout << "Expected: 3\n";

}

/*

```

Question 9: Check if Two Strings are Anagrams

Problem: Check if two strings contain same characters with same frequencies

Approach: Sort both strings and compare

Time Complexity: $O(n \log n)$

Space Complexity: $O(1)$ - excluding space for sorting

```

*/

```

```

bool areAnagrams(string s1, string s2) {

    if(s1.length() != s2.length()) return false;

```

```

        sort(s1.begin(), s1.end());

        sort(s2.begin(), s2.end());


        return s1 == s2;
    }


void testQ9() {

    cout << "\n===== Question 9: Check Anagrams =====\n";

    string s1 = "listen", s2 = "silent";

    cout << "Input: " << s1 << ", " << s2 << "\n";

    cout << "Output: " << (areAnagrams(s1, s2) ? "true" : "false") << "\n";

    cout << "Expected: true\n";

}

/*

```

Question 10: Find Intersection of Two Arrays

Problem: Find common elements between two arrays

Approach: Use two pointers on sorted arrays

Time Complexity: $O(n + m)$

Space Complexity: $O(\min(n, m))$

```

*/

vector<int> findIntersection(int arr1[], int n1, int arr2[], int n2) {

    vector<int> result;

    int i = 0, j = 0;

```

```

while(i < n1 && j < n2) {

    if(arr1[i] < arr2[j]) {

        i++;

    }

    else if(arr1[i] > arr2[j]) {

        j++;

    }

    else {

        result.push_back(arr1[i]);

        i++;

        j++;

    }

}

return result;

}

void testQ10() {

    cout << "\n===== Question 10: Array Intersection =====\n";

    int arr1[] = {1, 2, 3, 4, 5};

    int arr2[] = {3, 4, 5, 6, 7};

    cout << "Input: [1, 2, 3, 4, 5], [3, 4, 5, 6, 7]\n";

    vector<int> result = findIntersection(arr1, 5, arr2, 5);

```

```

    cout << "Output: [";

    for(int i = 0; i < result.size(); i++) {

        cout << result[i];

        if(i < result.size()-1) cout << ", ";

    }

    cout << "]\n";

    cout << "Expected: [3, 4, 5]\n";

}

// ===== SECTION 2: SEARCHING & SORTING =====

/*

```

Question 11: Binary Search

Problem: Search for target element in sorted array

Approach: Divide and conquer - compare with middle element

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

*/

```

int binarySearch(int arr[], int n, int target) {

    int left = 0, right = n - 1;

    while(left <= right) {

        int mid = left + (right - left) / 2;

        if(arr[mid] == target) {

```

```

        return mid;

    }

    else if(arr[mid] < target) {

        left = mid + 1;

    }

    else {

        right = mid - 1;

    }

}

return -1;

}

void testQ11() {

    cout << "\n===== Question 11: Binary Search =====\n";

    int arr[] = {1, 3, 5, 7, 9, 11, 13, 15};

    int n = 8, target = 7;

    cout << "Input: [1, 3, 5, 7, 9, 11, 13, 15], target=7\n";

    cout << "Output: " << binarySearch(arr, n, target) << "\n";

    cout << "Expected: 3\n";

}

```

Question 12: Bubble Sort

Problem: Sort array using bubble sort algorithm

Approach: Repeatedly swap adjacent elements if in wrong order

Time Complexity: $O(n^2)$ worst/average, $O(n)$ best

Space Complexity: $O(1)$

*/

```
void bubbleSort(int arr[], int n) {  
    for(int i = 0; i < n - 1; i++) {  
        bool swapped = false;  
  
        for(int j = 0; j < n - i - 1; j++) {  
            if(arr[j] > arr[j + 1]) {  
                swap(arr[j], arr[j + 1]);  
                swapped = true;  
            }  
        }  
  
        if(!swapped) break; // Optimization  
    }  
}  
  
void testQ12() {  
    cout << "\n===== Question 12: Bubble Sort =====\n";  
  
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
```



```

int n = 7;

cout << "Input: [64, 34, 25, 12, 22, 11, 90]\n";

bubbleSort(arr, n);

cout << "Output: [";

for(int i = 0; i < n; i++) {

    cout << arr[i];

    if(i < n-1) cout << ", ";

}

cout << "]\n";

cout << "Expected: [11, 12, 22, 25, 34, 64, 90]\n";

}

/*

```

Question 13: Selection Sort

Problem: Sort array using selection sort algorithm

Approach: Select minimum element and place at beginning

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

*/

```

void selectionSort(int arr[], int n) {

    for(int i = 0; i < n - 1; i++) {

        int minIdx = i;

        for(int j = i + 1; j < n; j++) {

```

```

        if(arr[j] < arr[minIdx]) {

            minIdx = j;

        }

    }

    swap(arr[i], arr[minIdx]);

}

}

void testQ13() {

    cout << "\n===== Question 13: Selection Sort =====\n";

    int arr[] = {64, 25, 12, 22, 11};

    int n = 5;

    cout << "Input: [64, 25, 12, 22, 11]\n";

    selectionSort(arr, n);

    cout << "Output: [";

    for(int i = 0; i < n; i++) {

        cout << arr[i];

        if(i < n-1) cout << ", ";

    }

    cout << "]\n";

    cout << "Expected: [11, 12, 22, 25, 64]\n";

}

```

```
/*
```

Question 14: Insertion Sort

Problem: Sort array using insertion sort algorithm

Approach: Build sorted array one element at a time

Time Complexity: $O(n^2)$ worst/average, $O(n)$ best

Space Complexity: $O(1)$

```
*/
```

```
void insertionSort(int arr[], int n) {
```

```
    for(int i = 1; i < n; i++) {
```

```
        int key = arr[i];
```

```
        int j = i - 1;
```

```
        while(j >= 0 && arr[j] > key) {
```

```
            arr[j + 1] = arr[j];
```

```
            j--;
```

```
        }
```

```
        arr[j + 1] = key;
```

```
    }
```

```
}
```

```
void testQ14() {
```

```
    cout << "\n===== Question 14: Insertion Sort =====\n";
```

```

int arr[] = {12, 11, 13, 5, 6};

int n = 5;

cout << "Input: [12, 11, 13, 5, 6]\n";

insertionSort(arr, n);

cout << "Output: [";

for(int i = 0; i < n; i++) {

    cout << arr[i];

    if(i < n-1) cout << ", ";

}

cout << "]\n";

cout << "Expected: [5, 6, 11, 12, 13]\n";

}

/*

```

Question 15: Find First and Last Position

Problem: Find first and last occurrence of target in array

Approach: Linear scan or binary search variants

Time Complexity: $O(n)$ linear, $O(\log n)$ binary

Space Complexity: $O(1)$

```

*/

pair<int, int> findFirstLast(int arr[], int n, int target) {

    int first = -1, last = -1;

    for(int i = 0; i < n; i++) {

```

```

        if(arr[i] == target) {

            if(first == -1) first = i;

            last = i;

        }

    }

    return {first, last};

}

void testQ15() {

    cout << "\n===== Question 15: First & Last Position =====\n";

    int arr[] = {1, 2, 3, 3, 3, 4, 5};

    int n = 7, target = 3;

    cout << "Input: [1, 2, 3, 3, 3, 4, 5], target=3\n";

    pair<int, int> result = findFirstLast(arr, n, target);

    cout << "Output: (" << result.first << ", " << result.second << ")\n";

    cout << "Expected: (2, 4)\n";

}

```

// ===== **SECTION 3: MATHEMATICAL PROBLEMS** =====

/*

Question 16: Check if Number is Prime

Problem: Determine if a number is prime

Approach: Check divisibility up to $\sqrt{n}$

Time Complexity: $O(\sqrt{n})$

Space Complexity: $O(1)$

*/

```
bool isPrime(int n) {  
    if(n <= 1) return false;  
    if(n <= 3) return true;  
    if(n % 2 == 0 || n % 3 == 0) return false;  
  
    for(int i = 5; i * i <= n; i += 6) {  
        if(n % i == 0 || n % (i + 2) == 0) {  
            return false;  
        }  
    }  
  
    return true;  
}
```

```
void testQ16() {  
    cout << "\n===== Question 16: Check Prime =====\n";  
    int n = 29;  
    cout << "Input: " << n << "\n";  
    cout << "Output: " << (isPrime(n) ? "true" : "false") << "\n";  
    cout << "Expected: true\n";  
}
```

/*

Question 17: Factorial

Problem: Calculate factorial of a number

Approach: Iterative multiplication or recursion

Time Complexity: $O(n)$

Space Complexity: $O(1)$ iterative, $O(n)$ recursive

*/

```
long long factorial(int n) {
```

```
    if(n < 0) return -1;
```

```
    if(n <= 1) return 1;
```

```
    long long result = 1;
```

```
    for(int i = 2; i <= n; i++) {
```

```
        result *= i;
```

```
    }
```

```
    return result;
```

```
}
```

```
long long factorialRecursive(int n) {
```

```
    if(n <= 1) return 1;
```

```
    return n * factorialRecursive(n - 1);
```

```
}
```

```
void testQ17() {
```

```
    cout << "\n===== Question 17: Factorial =====\n";
```

```
    int n = 5;
```

```
    cout << "Input: " << n << "\n";
```

```
    cout << "Output: " << factorial(n) << "\n";
```

```
    cout << "Expected: 120\n";
```

```
}
```

```
/*
```

Question 18: Fibonacci Number

Problem: Find nth Fibonacci number

Approach: Iterative approach for efficiency

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```
*/
```

```
int fibonacci(int n) {
```

```
    if(n <= 1) return n;
```

```
    int a = 0, b = 1, c;
```

```
    for(int i = 2; i <= n; i++) {
```

```
        c = a + b;
```



```

        a = b;

        b = c;

    }

    return b;

}

void testQ18() {

    cout << "\n===== Question 18: Fibonacci =====\n";

    int n = 10;

    cout << "Input: " << n << "\n";

    cout << "Output: " << fibonacci(n) << "\n";

    cout << "Expected: 55\n";

}

/*

```

Question 19: GCD (Greatest Common Divisor)

Problem: Find GCD of two numbers using Euclidean algorithm

Approach: Repeatedly apply $a = b$, $b = a \% b$ until $b = 0$

Time Complexity: $O(\log \min(a, b))$

Space Complexity: $O(1)$

*/

```

int gcd(int a, int b) {

    while(b != 0) {

```

```

        int temp = b;

        b = a % b;

        a = temp;

    }

    return a;

}

void testQ19() {

    cout << "\n===== Question 19: GCD =====\n";

    int a = 48, b = 18;

    cout << "Input: " << a << ", " << b << "\n";

    cout << "Output: " << gcd(a, b) << "\n";

    cout << "Expected: 6\n";

}

```

/*

Question 20: LCM (Least Common Multiple)

Problem: Find LCM of two numbers

Approach: Use formula $LCM(a,b) = (a*b) / GCD(a,b)$

Time Complexity: $O(\log \min(a,b))$

Space Complexity: $O(1)$

*/

```

int lcm(int a, int b) {

```

```

    return (a * b) / gcd(a, b);
}

void testQ20() {
    cout << "\n===== Question 20: LCM =====\n";

    int a = 12, b = 15;

    cout << "Input: " << a << ", " << b << "\n";

    cout << "Output: " << lcm(a, b) << "\n";

    cout << "Expected: 60\n";
}

```

/*

Question 21: Armstrong Number

Problem: Check if number equals sum of its digits raised to power of digit count

Example: $153 = 1^3 + 5^3 + 3^3 = 1 + 125 + 27$

Time Complexity: $O(d)$ where d is number of digits

Space Complexity: $O(1)$

*/

```

bool isArmstrong(int n) {
    int original = n;

    int digits = to_string(n).length();

    int sum = 0;

```

```

while(n > 0) {

    int digit = n % 10;

    sum += pow(digit, digits);

    n /= 10;

}

return sum == original;

}

void testQ21() {

    cout << "\n===== Question 21: Armstrong Number =====\n";

    int n = 153;

    cout << "Input: " << n << "\n";

    cout << "Output: " << (isArmstrong(n) ? "true" : "false") << "\n";

    cout << "Calculation:  $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$ \n";

    cout << "Expected: true\n";

}

/*

```

Question 22: Perfect Number

Problem: Check if number equals sum of its proper divisors

Example: $28 = 1 + 2 + 4 + 7 + 14$

Time Complexity: $O(\sqrt{n})$

Space Complexity: $O(1)$

```
*/
```

```
bool isPerfect(int n) {
```

```
    if(n <= 1) return false;
```

```
    int sum = 1;
```

```
    for(int i = 2; i * i <= n; i++) {
```

```
        if(n % i == 0) {
```

```
            sum += i;
```

```
            if(i != n / i) {
```

```
                sum += n / i;
```

```
            }
```

```
        }
```

```
    }
```

```
    return sum == n;
```

```
}
```

```
void testQ22() {
```

```
    cout << "\n===== Question 22: Perfect Number =====\n";
```

```
    int n = 28;
```

```
    cout << "Input: " << n << "\n";
```

```
    cout << "Output: " << (isPerfect(n) ? "true" : "false") << "\n";
```

```
    cout << "Divisors: 1, 2, 4, 7, 14 -> Sum = 28\n";
```

```
    cout << "Expected: true\n";  
}
```

```
/*
```

Question 23: Sum of Digits

Problem: Calculate sum of all digits in a number

Time Complexity: $O(d)$ where d is number of digits

Space Complexity: $O(1)$

```
*/
```

```
int sumOfDigits(int n) {
```

```
    n = abs(n);
```

```
    int sum = 0;
```

```
    while(n > 0) {
```

```
        sum += n % 10;
```

```
        n /= 10;
```

```
    }
```

```
    return sum;
```

```
}
```

```
void testQ23() {
```

```
    cout << "\n===== Question 23: Sum of Digits =====\n";
```

```
    int n = 12345;
```

```

    cout << "Input: " << n << "\n";

    cout << "Output: " << sumOfDigits(n) << "\n";

    cout << "Expected: 15\n";

}

/*

```

Question 24: Reverse a Number

Problem: Reverse the digits of a number

Time Complexity: $O(d)$ where d is number of digits

Space Complexity: $O(1)$

```

*/

int reverseNumber(int n) {

    int sign = (n < 0) ? -1 : 1;

    n = abs(n);

    int reversed = 0;

    while(n > 0) {

        reversed = reversed * 10 + n % 10;

        n /= 10;

    }

    return sign * reversed;

}

```

```

void testQ24() {

    cout << "\n===== Question 24: Reverse Number =====\n";

    int n = 12345;

    cout << "Input: " << n << "\n";

    cout << "Output: " << reverseNumber(n) << "\n";

    cout << "Expected: 54321\n";

}

```

```

/*

```

Question 25: Palindrome Number

Problem: Check if number reads same forwards and backwards

Time Complexity: $O(d)$ where d is number of digits

Space Complexity: $O(1)$

```

*/

```

```

bool isPalindromeNumber(int n) {

    if(n < 0) return false;

    return n == reverseNumber(n);

}

```

```

void testQ25() {

    cout << "\n===== Question 25: Palindrome Number =====\n";

    int n = 12321;

    cout << "Input: " << n << "\n";

    cout << "Output: " << (isPalindromeNumber(n) ? "true" : "false") << "\n";

}

```



```
    cout << "Expected: true\n";  
}
```

```
// ===== SECTION 4: PATTERN PRINTING =====
```

```
/*
```

Question 26-30: Pattern Printing

Time Complexity: $O(n^2)$ for all patterns

Space Complexity: $O(1)$

```
*/
```

```
void testQ26() {  
    cout << "\n===== Question 26: Right Triangle =====\n";  
    int n = 4;  
    cout << "Input: n=" << n << "\n";  
    cout << "Output:\n";  
    for(int i = 1; i <= n; i++) {  
        for(int j = 1; j <= i; j++) {  
            cout << "* ";  
        }  
        cout << "\n";  
    }  
}
```

```
void testQ27() {
```

```

cout << "\n===== Question 27: Pyramid Pattern =====\n";

int n = 4;

cout << "Input: n=" << n << "\n";

cout << "Output:\n";

for(int i = 1; i <= n; i++) {

    for(int j = 1; j <= n - i; j++) cout << " ";

    for(int j = 1; j <= 2 * i - 1; j++) cout << "*";

    cout << "\n";

}

}

```

```

void testQ28() {

    cout << "\n===== Question 28: Number Pyramid =====\n";

    int n = 4;

    cout << "Input: n=" << n << "\n";

    cout << "Output:\n";

    for(int i = 1; i <= n; i++) {

        for(int j = 1; j <= i; j++) {

            cout << j << " ";

        }

        cout << "\n";

    }

}

```

```

void testQ29() {

    cout << "\n===== Question 29: Diamond Pattern =====\n";

    int n = 4;

    cout << "Input: n=" << n << "\n";

    cout << "Output:\n";

    // Upper half
    for(int i = 1; i <= n; i++) {

        for(int j = 1; j <= n - i; j++) cout << " ";

        for(int j = 1; j <= 2 * i - 1; j++) cout << "*";

        cout << "\n";

    }

    // Lower half
    for(int i = n - 1; i >= 1; i--) {

        for(int j = 1; j <= n - i; j++) cout << " ";

        for(int j = 1; j <= 2 * i - 1; j++) cout << "*";

        cout << "\n";

    }

}

```

```

void testQ30() {

    cout << "\n===== Question 30: Hollow Square =====\n";

    int n = 5;

    cout << "Input: n=" << n << "\n";

    cout << "Output:\n";

```

```

for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= n; j++) {
        if(i == 1 || i == n || j == 1 || j == n)
            cout << "* ";
        else
            cout << " ";
    }
    cout << "\n";
}
}

```

// ===== SECTION 5: RECURSION =====

/*

Question 31: Power Function

Problem: Calculate $\text{base}^{\text{exponent}}$ using recursion

Time Complexity: $O(n)$ naive, $O(\log n)$ optimized

Space Complexity: $O(n)$ naive, $O(\log n)$ optimized

*/

```

int power(int base, int exp) {
    if(exp == 0) return 1;
    if(exp < 0) return 0; // For integer division
    return base * power(base, exp - 1);
}

```

```

// Optimized version using fast exponentiation

int powerOptimized(int base, int exp) {
    if(exp == 0) return 1;

    int half = powerOptimized(base, exp / 2);

    if(exp % 2 == 0)
        return half * half;
    else
        return base * half * half;
}

void testQ31() {
    cout << "\n===== Question 31: Power Function =====\n";

    int base = 2, exp = 10;

    cout << "Input: base=" << base << ", exp=" << exp << "\n";

    cout << "Output: " << power(base, exp) << "\n";

    cout << "Expected: 1024\n";

}

/*

```

Question 32: Sum of Natural Numbers

Problem: Calculate sum of first n natural numbers using recursion

Time Complexity: $O(n)$

Space Complexity: $O(n)$

*/

```
int sumNatural(int n) {  
    if(n <= 0) return 0;  
    return n + sumNatural(n - 1);  
}
```

```
void testQ32() {  
    cout << "\n===== Question 32: Sum of Natural Numbers =====\n";  
    int n = 10;  
    cout << "Input: " << n << "\n";  
    cout << "Output: " << sumNatural(n) << "\n";  
    cout << "Expected: 55\n";  
}
```

/*

Question 33: Print Numbers Using Recursion

Problem: Print numbers from 1 to n using recursion

Time Complexity: $O(n)$

Space Complexity: $O(n)$

*/

```
void printNumbers(int n) {  
    if(n > 0) {  
        printNumbers(n - 1);
```

```

        cout << n << " ";

    }

}

void testQ33() {

    cout << "\n===== Question 33: Print Numbers =====\n";

    int n = 10;

    cout << "Input: n=" << n << "\n";

    cout << "Output: ";

    printNumbers(n);

    cout << "\nExpected: 1 2 3 4 5 6 7 8 9 10\n";

}

```

```

/*

```

Question 34: Tower of Hanoi

Problem: Solve Tower of Hanoi puzzle

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$

```

*/

```

```

void towerOfHanoi(int n, char from, char to, char aux) {

    if(n == 1) {

        cout << "Move disk 1 from " << from << " to " << to << "\n";

        return;

    }
}

```

```

towerOfHanoi(n - 1, from, aux, to);

cout << "Move disk " << n << " from " << from << " to " << to << "\n";

towerOfHanoi(n - 1, aux, to, from);

}

void testQ34() {

    cout << "\n===== Question 34: Tower of Hanoi =====\n";

    int n = 3;

    cout << "Input: n=" << n << "\n";

    cout << "Output:\n";

    towerOfHanoi(n, 'A', 'C', 'B');

}

```

/*

Question 35: Count Digits Using Recursion

Problem: Count number of digits in a number

Time Complexity: $O(d)$ where d is number of digits

Space Complexity: $O(d)$

*/

```

int countDigits(int n) {

    if(n == 0) return 0;

    return 1 + countDigits(n / 10);

}

```



```

void testQ35() {

    cout << "\n===== Question 35: Count Digits =====\n";

    int n = 12345;

    cout << "Input: " << n << "\n";

    cout << "Output: " << (n == 0 ? 1 : countDigits(n)) << "\n";

    cout << "Expected: 5\n";

}

```

```

// ===== SECTION 6: STRING MANIPULATION =====

```

```

/*

```

Question 36: Convert to Uppercase

Problem: Convert all characters in string to uppercase

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```

*/

```

```

string toUpperCase(string s) {

    for(char &c : s) {

        c = toupper(c);

    }

    return s;

}

```

```

void testQ36() {

```

```

cout << "\n===== Question 36: Convert to Uppercase =====\n";

string s = "Hello World!";

cout << "Input: " << s << "\n";

cout << "Output: " << toUpperCase(s) << "\n";

cout << "Expected: 'HELLO WORLD!'\n";

}

```

```

/*

```

Question 37: Count Words in String

Problem: Count number of words in a string

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```

*/

```

```

int countWords(string s) {

    int count = 0;

    bool inWord = false;

    for(char c : s) {

        if(isspace(c)) {

            inWord = false;

        }

        else if(!inWord) {

            inWord = true;

            count++;

        }

    }

}

```

```

    }

}

return count;

}

void testQ37() {

    cout << "\n===== Question 37: Count Words =====\n";

    string s = "The quick brown fox jumps";

    cout << "Input: '" << s << "'\n";

    cout << "Output: " << countWords(s) << "\n";

    cout << "Expected: 5\n";

}

/*

```

Question 38: Remove Spaces from String

Problem: Remove all spaces from a string

Time Complexity: $O(n)$

Space Complexity: $O(n)$

```

*/

string removeSpaces(string s) {

    string result = "";

    for(char c : s) {

        if(!isspace(c)) {

```

```

        result += c;
    }
}

return result;
}

void testQ38() {
    cout << "\n===== Question 38: Remove Spaces =====\n";
    string s = "Hello World Program";
    cout << "Input: " << s << "\n";
    cout << "Output: " << removeSpaces(s) << "\n";
    cout << "Expected: 'HelloWorldProgram'\n";
}

/*

```

Question 39: Character Frequency

Problem: Find frequency of each character in string

Time Complexity: $O(n)$

Space Complexity: $O(1)$ - fixed size array

*/

```

void charFrequency(string s) {
    unordered_map<char, int> freq;

    for(char c : s) {

```

```

        freq[c]++;
    }

    for(auto &pair : freq) {
        cout << "" << pair.first << ": " << pair.second << "\n";
    }
}

void testQ39() {
    cout << "\n===== Question 39: Character Frequency =====\n";
    string s = "hello";
    cout << "Input: " << s << "\n";
    cout << "Output:\n";
    charFrequency(s);
}

```

```

/*

```

Question 40: Check if String is Numeric

Problem: Check if string contains only digits

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```

*/

```

```

bool isNumeric(string s) {
    if(s.empty()) return false;

```

```

    for(char c : s) {
        if(!isdigit(c)) {
            return false;
        }
    }

    return true;
}

void testQ40() {
    cout << "\n===== Question 40: Check Numeric String =====\n";
    string s = "12345";
    cout << "Input: " << s << "\n";
    cout << "Output: " << (isNumeric(s) ? "true" : "false") << "\n";
    cout << "Expected: true\n";
}

```

// ===== **SECTION 7: ADVANCED ARRAY PROBLEMS** =====

/*

Question 41: Linear Search

Problem: Find index of target element in array

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```

*/

int linearSearch(int arr[], int n, int target) {

    for(int i = 0; i < n; i++) {

        if(arr[i] == target) {

            return i;

        }

    }

    return -1;

}

void testQ41() {

    cout << "\n===== Question 41: Linear Search =====\n";

    int arr[] = {10, 20, 30, 40, 50};

    int n = 5, target = 30;

    cout << "Input: [10, 20, 30, 40, 50], target=30\n";

    cout << "Output: " << linearSearch(arr, n, target) << "\n";

    cout << "Expected: 2\n";

}

/*

```

Question 42: Count Element Frequency

Problem: Count frequency of each element in array

Time Complexity: $O(n)$

Space Complexity: $O(n)$

```
*/
```

```
void countFrequency(int arr[], int n) {
```

```
    unordered_map<int, int> freq;
```

```
    for(int i = 0; i < n; i++) {
```

```
        freq[arr[i]]++;
```

```
    }
```

```
    for(auto &pair : freq) {
```

```
        cout << pair.first << " occurs " << pair.second << " times\n";
```

```
    }
```

```
}
```

```
void testQ42() {
```

```
    cout << "\n===== Question 42: Element Frequency =====\n";
```

```
    int arr[] = {1, 2, 2, 3, 3, 3, 4};
```

```
    int n = 7;
```

```
    cout << "Input: [1, 2, 2, 3, 3, 3, 4]\n";
```

```
    cout << "Output:\n";
```

```
    countFrequency(arr, n);
```

```
}
```

```
/*
```


Question 43: Find Majority Element

Problem: Find element that appears more than $n/2$ times

Approach: Boyer-Moore Voting Algorithm

Time Complexity: $O(n)$

Space Complexity: $O(1)$

*/

```
int findMajority(int arr[], int n) {  
    int candidate = arr[0], count = 1;  
  
    // Find candidate  
    for(int i = 1; i < n; i++) {  
        if(arr[i] == candidate) {  
            count++;  
        }  
        else {  
            count--;  
        }  
  
        if(count == 0) {  
            candidate = arr[i];  
            count = 1;  
        }  
    }  
}
```

```

// Verify candidate

count = 0;

for(int i = 0; i < n; i++) {
    if(arr[i] == candidate) {
        count++;
    }
}

return (count > n / 2) ? candidate : -1;
}

void testQ43() {
    cout << "\n===== Question 43: Majority Element =====\n";

    int arr[] = {2, 2, 1, 1, 2, 2, 2};

    int n = 7;

    cout << "Input: [2, 2, 1, 1, 2, 2, 2]\n";

    cout << "Output: " << findMajority(arr, n) << "\n";

    cout << "Expected: 2\n";

}

/*

```

Question 44: Check if Array is Sorted

Problem: Check if array is sorted in ascending order

Time Complexity: $O(n)$

Space Complexity: $O(1)$

*/

```
bool isSorted(int arr[], int n) {  
    for(int i = 1; i < n; i++) {  
        if(arr[i] < arr[i - 1]) {  
            return false;  
        }  
    }  
    return true;  
}
```

```
void testQ44() {  
    cout << "\n===== Question 44: Check Sorted Array =====\n";  
    int arr[] = {1, 2, 3, 4, 5};  
    int n = 5;  
    cout << "Input: [1, 2, 3, 4, 5]\n";  
    cout << "Output: " << (isSorted(arr, n) ? "true" : "false") << "\n";  
    cout << "Expected: true\n";  
}
```

/*

Question 45: Find Leaders in Array

Problem: Element is leader if greater than all elements to its right

Time Complexity: $O(n)$

Space Complexity: $O(n)$

*/

```
vector<int> findLeaders(int arr[], int n) {
```

```
    vector<int> leaders;
```

```
    int maxRight = INT_MIN;
```

```
    for(int i = n - 1; i >= 0; i--) {
```

```
        if(arr[i] > maxRight) {
```

```
            leaders.push_back(arr[i]);
```

```
            maxRight = arr[i];
```

```
        }
```

```
    }
```

```
    reverse(leaders.begin(), leaders.end());
```

```
    return leaders;
```

```
}
```

```
void testQ45() {
```

```
    cout << "\n===== Question 45: Find Leaders =====\n";
```

```
    int arr[] = {16, 17, 4, 3, 5, 2};
```

```
    int n = 6;
```

```
    cout << "Input: [16, 17, 4, 3, 5, 2]\n";
```

```
    vector<int> leaders = findLeaders(arr, n);
```

```
    cout << "Output: [";
```

```

for(int i = 0; i < leaders.size(); i++) {

    cout << leaders[i];

    if(i < leaders.size()-1) cout << ", ";

}

cout << "]\n";

cout << "Expected: [17, 5, 2]\n";

}

/*

```

Question 46: Merge Two Sorted Arrays

Problem: Merge two sorted arrays into one sorted array

Time Complexity: $O(n + m)$

Space Complexity: $O(n + m)$

```

*/

vector<int> mergeSortedArrays(int arr1[], int n1, int arr2[], int n2) {

    vector<int> result;

    int i = 0, j = 0;

    while(i < n1 && j < n2) {

        if(arr1[i] < arr2[j]) {

            result.push_back(arr1[i++]);

        }

        else {

            result.push_back(arr2[j++]);

        }

    }

}

```

```

    }
}

while(i < n1) result.push_back(arr1[i++]);
while(j < n2) result.push_back(arr2[j++]);

return result;
}

void testQ46() {
    cout << "\n===== Question 46: Merge Sorted Arrays =====\n";

    int arr1[] = {1, 3, 5, 7};
    int arr2[] = {2, 4, 6, 8};

    cout << "Input: [1, 3, 5, 7], [2, 4, 6, 8]\n";

    vector<int> result = mergeSortedArrays(arr1, 4, arr2, 4);

    cout << "Output: [";

    for(int i = 0; i < result.size(); i++) {
        cout << result[i];

        if(i < result.size()-1) cout << ", ";
    }

    cout << "]\n";

    cout << "Expected: [1, 2, 3, 4, 5, 6, 7, 8]\n";
}

```

/*

Question 47: Find Equilibrium Index

Problem: Find index where sum of left elements equals sum of right

Time Complexity: $O(n)$

Space Complexity: $O(1)$

*/

```
int findEquilibrium(int arr[], int n) {
```

```
    int totalSum = 0, leftSum = 0;
```

```
    for(int i = 0; i < n; i++) {
```

```
        totalSum += arr[i];
```

```
    }
```

```
    for(int i = 0; i < n; i++) {
```

```
        totalSum -= arr[i];
```

```
        if(leftSum == totalSum) {
```

```
            return i;
```

```
        }
```

```
        leftSum += arr[i];
```

```
    }
```

```

    return -1;
}

void testQ47() {
    cout << "\n===== Question 47: Equilibrium Index =====\n";

    int arr[] = {1, 3, 5, 2, 2};

    int n = 5;

    cout << "Input: [1, 3, 5, 2, 2]\n";

    cout << "Output: " << findEquilibrium(arr, n) << "\n";

    cout << "Expected: 2 (left sum: 1+3=4, right sum: 2+2=4)\n";
}

```

```

/*

```

Question 48: Find Pairs with Given Sum

Problem: Find all pairs in array that sum to target

Time Complexity: $O(n^2)$ brute force, $O(n)$ with hash map

Space Complexity: $O(1)$ brute force, $O(n)$ with hash map

```

*/

```

```

void findPairs(int arr[], int n, int target) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = i + 1; j < n; j++) {
            if(arr[i] + arr[j] == target) {
                cout << "(" << arr[i] << ", " << arr[j] << ") \n";
            }
        }
    }
}

```



```

    }

}

}

void testQ48() {

    cout << "\n===== Question 48: Pairs with Sum =====\n";

    int arr[] = {1, 2, 3, 4, 5};

    int n = 5, target = 6;

    cout << "Input: [1, 2, 3, 4, 5], target=6\n";

    cout << "Output:\n";

    findPairs(arr, n, target);

    cout << "Expected: (1, 5), (2, 4)\n";

}

```

```

/*

```

Question 49: Check Power of 2

Problem: Check if number is a power of 2

Approach: Use bit manipulation - power of 2 has only one set bit

Time Complexity: O(1)

Space Complexity: O(1)

```

*/

```

```

bool isPowerOf2(int n) {

    return n > 0 && (n & (n - 1)) == 0;

}

```

```

void testQ49() {

    cout << "\n===== Question 49: Power of 2 =====\n";

    int n = 16;

    cout << "Input: " << n << "\n";

    cout << "Output: " << (isPowerOf2(n) ? "true" : "false") << "\n";

    cout << "Expected: true\n";

}

```

```

/*

```

Question 50: Count Set Bits

Problem: Count number of 1s in binary representation

Approach: Use bit manipulation

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

```

*/

```

```

int countSetBits(int n) {

    int count = 0;

    while(n) {

        count += n & 1;

        n >>= 1;

    }

    return count;

}

```

```

void testQ50() {

    cout << "\n===== Question 50: Count Set Bits =====\n";

    int n = 15; // Binary: 1111

    cout << "Input: " << n << " (Binary: 1111)\n";

    cout << "Output: " << countSetBits(n) << "\n";

    cout << "Expected: 4\n";

}

```

// ===== **BONUS QUESTIONS** =====

/*

Question 51: Move Zeros to End

Problem: Move all zeros to end maintaining order of non-zero elements

Time Complexity: $O(n)$

Space Complexity: $O(1)$

*/

```

void moveZerosToEnd(int arr[], int n) {

    int nonZeroPos = 0;

    for(int i = 0; i < n; i++) {

        if(arr[i] != 0) {

            arr[nonZeroPos++] = arr[i];

        }

    }

}

```

```

while(nonZeroPos < n) {
    arr[nonZeroPos++] = 0;
}
}

void testQ51() {
    cout << "\n===== Question 51: Move Zeros to End =====\n";
    int arr[] = {0, 1, 0, 3, 12};
    int n = 5;
    cout << "Input: [0, 1, 0, 3, 12]\n";
    moveZerosToEnd(arr, n);
    cout << "Output: [";
    for(int i = 0; i < n; i++) {
        cout << arr[i];
        if(i < n-1) cout << ", ";
    }
    cout << "]\n";
    cout << "Expected: [1, 3, 12, 0, 0]\n";
}

/*

```

Question 52: Find Duplicate in Array

Problem: Find duplicate element in array of $n+1$ integers (1 to n)

Time Complexity: $O(n)$

Space Complexity: $O(1)$

*/

```
int findDuplicate(int arr[], int n) {
```

```
    // Using Floyd's Cycle Detection
```

```
    int slow = arr[0];
```

```
    int fast = arr[0];
```

```
    do {
```

```
        slow = arr[slow];
```

```
        fast = arr[arr[fast]];
```

```
    } while(slow != fast);
```

```
    slow = arr[0];
```

```
    while(slow != fast) {
```

```
        slow = arr[slow];
```

```
        fast = arr[fast];
```

```
    }
```

```
    return slow;
```

```
}
```

```
void testQ52() {
```

```
    cout << "\n===== Question 52: Find Duplicate =====\n";
```

```

int arr[] = {1, 3, 4, 2, 2};

int n = 5;

cout << "Input: [1, 3, 4, 2, 2]\n";

cout << "Output: " << findDuplicate(arr, n) << "\n";

cout << "Expected: 2\n";

}

/*

```

Question 53: Kadane's Algorithm - Maximum Subarray Sum

Problem: Find maximum sum of contiguous subarray

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```

*/

int maxSubarraySum(int arr[], int n) {

    int maxSum = arr[0];

    int currentSum = arr[0];

    for(int i = 1; i < n; i++) {

        currentSum = max(arr[i], currentSum + arr[i]);

        maxSum = max(maxSum, currentSum);

    }

    return maxSum;

}

```

```

void testQ53() {

    cout << "\n===== Question 53: Maximum Subarray Sum =====\n";

    int arr[] = {-2, 1, -3, 4, -1, 2, 1, -5, 4};

    int n = 9;

    cout << "Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4]\n";

    cout << "Output: " << maxSubarraySum(arr, n) << "\n";

    cout << "Expected: 6 (subarray: [4, -1, 2, 1])\n";

}

/*

```

Question 54: Stock Buy Sell Problem

Problem: Find maximum profit from one buy and one sell

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```

*/

int maxProfit(int prices[], int n) {

    int minPrice = INT_MAX;

    int maxProfit = 0;

    for(int i = 0; i < n; i++) {

        minPrice = min(minPrice, prices[i]);

        maxProfit = max(maxProfit, prices[i] - minPrice);

    }

}

```

```

        return maxProfit;
    }

void testQ54() {
    cout << "\n===== Question 54: Stock Buy Sell =====\n";

    int prices[] = {7, 1, 5, 3, 6, 4};

    int n = 6;

    cout << "Input: [7, 1, 5, 3, 6, 4]\n";

    cout << "Output: " << maxProfit(prices, n) << "\n";

    cout << "Expected: 5 (buy at 1, sell at 6)\n";

}

/*

```

Question 55: Two Sum Problem

Problem: Find two numbers that add up to target

Time Complexity: $O(n)$

Space Complexity: $O(n)$

```

*/

pair<int, int> twoSum(int arr[], int n, int target) {
    unordered_map<int, int> map;

    for(int i = 0; i < n; i++) {
        int complement = target - arr[i];
    }
}

```



```

        if(map.find(complement) != map.end()) {

            return {map[complement], i};

        }

        map[arr[i]] = i;

    }

    return {-1, -1};

}

void testQ55() {

    cout << "\n===== Question 55: Two Sum =====\n";

    int arr[] = {2, 7, 11, 15};

    int n = 4, target = 9;

    cout << "Input: [2, 7, 11, 15], target=9\n";

    pair<int, int> result = twoSum(arr, n, target);

    cout << "Output: (" << result.first << ", " << result.second << ")\n";

    cout << "Expected: (0, 1)\n";

}

/*

```

Question 56: Product of Array Except Self

Problem: Return array where output[i] = product of all except arr[i]

Time Complexity: $O(n)$

Space Complexity: $O(1)$ - excluding output array

*/

```
vector<int> productExceptSelf(int arr[], int n) {
```

```
    vector<int> result(n, 1);
```

```
    // Left products
```

```
    int left = 1;
```

```
    for(int i = 0; i < n; i++) {
```

```
        result[i] = left;
```

```
        left *= arr[i];
```

```
    }
```

```
    // Right products
```

```
    int right = 1;
```

```
    for(int i = n - 1; i >= 0; i--) {
```

```
        result[i] *= right;
```

```
        right *= arr[i];
```

```
    }
```

```
    return result;
```

```
}
```

```
void testQ56() {
```

```

cout << "\n===== Question 56: Product Except Self =====\n";

int arr[] = {1, 2, 3, 4};

int n = 4;

cout << "Input: [1, 2, 3, 4]\n";

vector<int> result = productExceptSelf(arr, n);

cout << "Output: [";

for(int i = 0; i < result.size(); i++) {

    cout << result[i];

    if(i < result.size()-1) cout << ", ";

}

cout << "]\n";

cout << "Expected: [24, 12, 8, 6]\n";

}

/*

```

Question 57: Longest Consecutive Sequence

Problem: Find length of longest consecutive elements sequence

Time Complexity: $O(n)$

Space Complexity: $O(n)$

*/

```

int longestConsecutive(int arr[], int n) {

    unordered_set<int> numSet(arr, arr + n);

    int longest = 0;

```

```

for(int num : numSet) {

    if(numSet.find(num - 1) == numSet.end()) {

        int current = num;

        int streak = 1;


        while(numSet.find(current + 1) != numSet.end()) {

            current++;

            streak++;

        }


        longest = max(longest, streak);

    }

}


return longest;

}


void testQ57() {

    cout << "\n===== Question 57: Longest Consecutive Sequence =====\n";

    int arr[] = {100, 4, 200, 1, 3, 2};

    int n = 6;

    cout << "Input: [100, 4, 200, 1, 3, 2]\n";

    cout << "Output: " << longestConsecutive(arr, n) << "\n";

    cout << "Expected: 4 (sequence: 1, 2, 3, 4)\n";

```

```
}
```

```
/*
```

Question 58: Container With Most Water

Problem: Find two lines that together with x-axis form container with most water

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```
*/
```

```
int maxArea(int height[], int n) {  
    int left = 0, right = n - 1;  
    int maxWater = 0;  
  
    while(left < right) {  
        int width = right - left;  
        int h = min(height[left], height[right]);  
        maxWater = max(maxWater, width * h);  
  
        if(height[left] < height[right]) {  
            left++;  
        }  
        else {  
            right--;  
        }  
    }  
}
```

```

    }

    return maxWater;
}

void testQ58() {
    cout << "\n===== Question 58: Container With Most Water =====\n";

    int height[] = {1, 8, 6, 2, 5, 4, 8, 3, 7};

    int n = 9;

    cout << "Input: [1, 8, 6, 2, 5, 4, 8, 3, 7]\n";

    cout << "Output: " << maxArea(height, n) << "\n";

    cout << "Expected: 49\n";
}

/*

```

Question 59: Sort Colors (Dutch National Flag)

Problem: Sort array containing only 0s, 1s, and 2s

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```

*/

```

```

void sortColors(int arr[], int n) {

    int low = 0, mid = 0, high = n - 1;

    while(mid <= high) {

```

```

        if(arr[mid] == 0) {

            swap(arr[low++], arr[mid++]);

        }

        else if(arr[mid] == 1) {

            mid++;

        }

        else {

            swap(arr[mid], arr[high--]);

        }

    }

}

```

```

void testQ59() {

    cout << "\n===== Question 59: Sort Colors =====\n";

    int arr[] = {2, 0, 2, 1, 1, 0};

    int n = 6;

    cout << "Input: [2, 0, 2, 1, 1, 0]\n";

    sortColors(arr, n);

    cout << "Output: [";

    for(int i = 0; i < n; i++) {

        cout << arr[i];

        if(i < n-1) cout << ", ";

    }

    cout << "]\n";
}

```

```

        cout << "Expected: [0, 0, 1, 1, 2, 2]\n";
    }

```

```

/*

```

Question 60: Find Kth Largest Element

Problem: Find the kth largest element in unsorted array

Approach: Use Quick Select algorithm or sort

Time Complexity: $O(n)$ average, $O(n^2)$ worst case

Space Complexity: $O(1)$

```

*/

```

```

int findKthLargest(int arr[], int n, int k) {
    // Simple approach: sort and return
    sort(arr, arr + n, greater<int>());
    return arr[k - 1];
}

```

```

void testQ60() {
    cout << "\n===== Question 60: Kth Largest Element =====\n";

    int arr[] = {3, 2, 1, 5, 6, 4};

    int n = 6, k = 2;

    cout << "Input: [3, 2, 1, 5, 6, 4], k=2\n";

    cout << "Output: " << findKthLargest(arr, n, k) << "\n";

    cout << "Expected: 5\n";
}

```


===== STUDY APPROACH =====

WEEK 1: Basic Array Operations (Q1-Q10)

- Practice array traversal and manipulation
- Master two-pointer technique
- Understand array rotation and reversal

WEEK 2: Searching & Sorting (Q11-Q15)

- Implement all sorting algorithms from scratch
- Practice binary search variations
- Analyze time complexity

WEEK 3: Mathematical Problems (Q16-Q25)

- Focus on number theory problems
- Practice digit manipulation
- Understand recursive vs iterative approaches

WEEK 4: Advanced Topics (Q26-Q60)

- Pattern printing for logical thinking
- Recursion and backtracking
- String manipulation techniques
- Advanced array algorithms

PRACTICE STRATEGY:

1. Understand the problem completely
2. Think of brute force solution first
3. Optimize using appropriate data structures
4. Consider edge cases
5. Analyze time and space complexity
6. Write clean, readable code
7. Test with multiple test cases

===== COMMON MISTAKES TO AVOID =====

1. Not handling edge cases (empty arrays, single element)
2. Integer overflow in calculations
3. Array index out of bounds
4. Not considering negative numbers
5. Forgetting to return values from functions
6. Not optimizing nested loops
7. Ignoring space complexity
8. Poor variable naming
9. Not testing with large inputs
10. Assuming sorted input when not specified

===== KEY CONCEPTS COVERED =====

- ✓ Array Manipulation & Traversal
- ✓ Two Pointer Technique
- ✓ Sliding Window
- ✓ Binary Search & Variants
- ✓ Sorting Algorithms (Bubble, Selection, Insertion)
- ✓ Recursion & Backtracking
- ✓ String Processing
- ✓ Mathematical Algorithms
- ✓ Bit Manipulation
- ✓ Hash Maps & Sets
- ✓ Dynamic Programming Basics
- ✓ Greedy Algorithms
- ✓ Pattern Recognition

===== RECOMMENDED PRACTICE ORDER =====

BEGINNERS:

Start with Q1-Q25 (Basic arrays, strings, and math)

INTERMEDIATE:

Focus on Q26-Q45 (Patterns, recursion, advanced arrays)

ADVANCED:

Master Q46-Q60 (Complex algorithms, optimization)

===== ADDITIONAL RESOURCES =====

Books:

- "Cracking the Coding Interview" by Gayle Laakmann McDowell
- "Introduction to Algorithms" by CLRS

Online Platforms:

- LeetCode (Easy to Medium problems)
- GeeksforGeeks (Company-specific questions)
- HackerRank (Practice contests)

YouTube Channels:

- Abdul Bari (Algorithms)
- Take U Forward (Striver's SDE Sheet)

===== FINAL TIPS =====

1. Practice typing code without IDE assistance
2. Time yourself while solving problems
3. Explain your solution out loud (mock interviews)
4. Review solutions even after solving correctly

5. Learn from multiple approaches to same problem
 6. Focus on code readability and clarity
 7. Prepare explanation for your approach
 8. Practice on paper/whiteboard occasionally
 9. Stay calm during the actual interview
 10. Ask clarifying questions before coding
-

All the best for your Infosys interview! 🎯



About the Author

SYNTAX ERROR | Abhishek Rathor

Helping students crack their dream placements! 🚀

Connect with me:

- 📷 Instagram: [@code.abhii07](#)
 - 📺 YouTube: [SYNTAX ERROR](#)
-



Youve Got This! All The Best!



"Success is where preparation meets opportunity. You've prepared well - now go seize your opportunity!"